Structured Query Language (SQL)

Although we refer to the SQL language as a "query language," it can do much more than just query a database. It can define the structure of the data, modify data in the database, and specify security constraints

The SQL language has several parts:

- **Data-definition language (DDL):** The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.

- **Data-manipulation language (DML):** The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.

• Integrity: The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.

• View definition: The SQL DDL includes commands for defining views.

• Transaction control: SQL includes commands for specifying the beginning and end points of transactions.

• Embedded SQL and dynamic SQL: Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, and Java.

• Authorization: The SQL DDL includes commands for specifying access rights to relations and views.

SQL Data Definition

Domain Types in SQL

- **char**(n): A fixed-length character string with user-specified length n. The full form, **character**, can be used instead.
- **varchar**(n): A variable-length character string with user-specified maximum length n. The full form, **character varying**, is equivalent.
- **int**: An integer (a finite subset of the integers that is machine dependent). The full form, **integer**, is equivalent.
- **smallint**: A small integer (a machine-dependent subset of the integer type).
- **numeric**(p, d): A fixed-point number with user-specified precision. The number consists of p digits (plus a sign), and d of the p digits are to the right of the decimal point. Thus, **numeric**(3,1) allows 44.5 to be stored exactly, but neither 444.5 nor 0.32 can be stored exactly in a field of this type.
- **real, double precision**: Floating-point and double-precision floating-point numbers with machine-dependent precision.
- **float**(n): A floating-point number with precision of at least n digits.

name
Char(20)
Singh

name
varchar(20)
Singh

Each type may include a special value called the null value. A null value indicates an absent value that may exist but be unknown or that may not exist at all.

The char data type stores fixed-length strings. Consider, for example, an attribute A of type char(10). If we stored a string “Avi” in this attribute, seven spaces are appended to the string to make it 10 characters long. In contrast, if attribute B were of type varchar(10), and we stored “Avi” in attribute B, no spaces would be added.

Basic Schema Definition

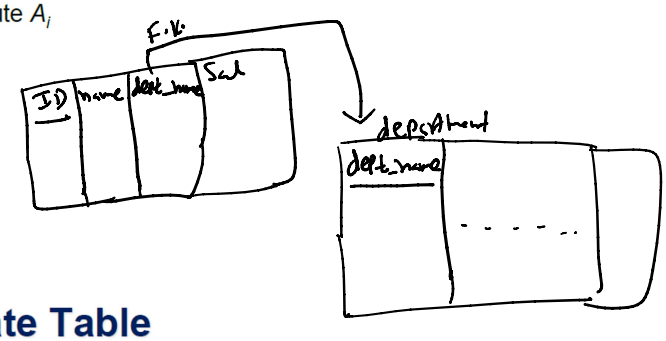
Create Table Construct

- An SQL relation is defined using the **create table** command:

  create table *r*
  (A_1 D_1, A_2 D_2, ..., A_n D_n,
  (integrity-constraint$_1$),
  ...,
  (integrity-constraint$_k$))

  - *r* is the name of the relation
  - each A_i is an attribute name in the schema of relation *r*
  - D_i is the data type of values in the domain of attribute A_i
- Example:

  create table *instructor* (
  ID **char**(5),
  name **varchar**(20),
  dept_name **varchar**(20),
  salary **numeric**(8,2))



Integrity Constraints in Create Table

- Types of integrity constraints
  - **primary key** (A_1, ..., A_n)
  - **foreign key** (A_m, ..., A_n) **references** *r*
  - **not null**
- SQL prevents any update to the database that violates an integrity constraint.
- Example:

  create table *instructor* (
  ID **char**(5),
  name **varchar**(20) **not null,**
  dept_name **varchar**(20),
  salary **numeric**(8,2),
  primary key (*ID*),
  foreign key (*dept_name*) **references** *department);*

```
create table department
    (dept_name      varchar (20),
     building       varchar (15),
     budget         numeric (12,2),
     primary key (dept_name));

create table course
    (course_id      varchar (7),
     title          varchar (50),
     dept_name      varchar (20),
     credits        numeric (2,0),
     primary key (course_id),
     foreign key (dept_name) references department);

create table instructor
    (ID             varchar (5),
     name           varchar (20) not null,
     dept_name      varchar (20),
     salary         numeric (8,2),
     primary key (ID),
     foreign key (dept_name) references department);

create table section
    (course_id      varchar (8),
     sec_id         varchar (8),
     semester       varchar (6),
     year           numeric (4,0),
     building       varchar (15),
     room_number    varchar (7),
     time_slot_id   varchar (4),
     primary key (course_id, sec_id, semester, year),
     foreign key (course_id) references course);

create table teaches
    (ID             varchar (5),
     course_id      varchar (8),
     sec_id         varchar (8),
     semester       varchar (6),
     year           numeric (4,0),
     primary key (ID, course_id, sec_id, semester, year),
     foreign key (course_id, sec_id, semester, year) references section,
     foreign key (ID) references instructor);
```

Figure SQL data definition for part of the university database.

- **primary key** $(A_{j_1}, A_{j_2}, \ldots, A_{j_m})$: The **primary-key** specification says that attributes $A_{j_1}, A_{j_2}, \ldots, A_{j_m}$ form the primary key for the relation. The primary-key attributes are required to be *nonnull* and *unique*; that is, no tuple can have a null value for a primary-key attribute, and no two tuples in the relation can be equal on all the primary-key attributes. Although the primary-key specification is optional, it is generally a good idea to specify a primary key for each relation.
- **foreign key** $(A_{k_1}, A_{k_2}, \ldots, A_{k_n})$ **references** s: The **foreign key** specification says that the values of attributes $(A_{k_1}, A_{k_2}, \ldots, A_{k_n})$ for any tuple in the relation must correspond to values of the primary key attributes of some tuple in relation s.
- **not null**: The **not null** constraint on an attribute specifies that the null value is not allowed for that attribute; in other words, the constraint excludes the null value from the domain of that attribute. For example, in Figure the **not null** constraint on the *name* attribute of the *instructor* relation ensures that the name of an instructor cannot be null.

A newly created relation is empty initially. Inserting tuples into a relation, updating them, and deleting them are done by data manipulation statements insert, update, and delete.

Updates to tables

- **Insert**
  - **insert into** *instructor* **values** ('10211', 'Smith', 'Biology', 66000);
- **Delete**
  - Remove all tuples from the *student* relation
    - **delete from** *student*
- **Drop Table**
  - **drop table** *r*
- **Alter**
  - **alter table** *r* **add** *A D*
    - where *A* is the name of the attribute to be added to relation *r* and *D* is the domain of *A*.
    - All exiting tuples in the relation are assigned *null* as the value for the new attribute.
  - **alter table** *r* **drop** *A*
    - where *A* is the name of an attribute of relation *r*
    - Dropping of attributes not supported by many databases.

To remove a relation from an SQL database, we use the **drop table** command. The drop table command deletes all information about the dropped relation from the database. The **drop table** command deletes not only all tuples of relation r, but also the schema for r.
The **delete from** command retains relation, but deletes all tuples in relation.

Data-Manipulation Language

- *Query* information from the database
- *Insert* tuples into the database
- *Delete* tuples from the database
- *Modify* tuples in the database

Basic Query Structure

- A typical SQL query has the form:

select $A_1, A_2, ..., A_n$
from $r_1, r_2, ..., r_m$
where P

$$\Pi_{A_1, A_2 \cdots A_n}\left(\sigma_P\left(r_1 \times r_2 \times r_3 \cdots r_m\right)\right)$$

  - A_i represents an attribute
  - R_i represents a relation
  - P is a predicate.
- The result of an SQL query is a relation.

$$\Pi_{AL}\left(\sigma_P\left(r_1 \times r_2\right)\right)$$

Select AL
from r_1, r_2
where P

The select Clause

- The **select** clause lists the attributes desired in the result of a query
  - corresponds to the projection operation of the relational algebra
- Example: find the names of all instructors:

select *name*
from *instructor*

$$\not\equiv \Pi_{name}(instructor)$$

Select distinct name from instructor

- NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)
  - E.g., *Name* ≡ *NAME* ≡ *name*
  - Some people use upper case wherever we use bold font.

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select.
- Find the department names of all instructors, and remove duplicates

select distinct *dept_name*
from *instructor*

dept_name
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

Result of "**select** *dept_name* **from** *instructor*".

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select.
- Find the department names of all instructors, and remove duplicates

```
select distinct dept_name
from instructor
```

- The keyword **all** specifies that duplicates should not be removed.

```
select all dept_name
from instructor
```

History
Finance
Biology
Comp. Sci.
Elec. Eng.

Result of "**select** *dept_name* **from** *instructor*".

Since duplicate retention is the default, we shall not use **all** in our examples. To ensure the elimination of duplicates in the results of our example queries, we shall use **distinct** whenever it is necessary.

- An asterisk in the select clause denotes "all attributes"

```
select *
from instructor
```

- The **select** clause can contain arithmetic expressions involving the operation, +, –, ∗, and /, and operating on constants or attributes of tuples.
  - The query:

```
select ID, name, salary/12
from instructor
```

    would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12.
  - Can rename "*salary/12*" using the **as** clause:

```
select ID, name, salary/12 as monthly_salary
```

instructor(<u>*ID*</u>, *name*, *dept_name*, *salary*)

- Find the information about all instructors and show their salaries with a hike of 10%.

```
select ID, name, dept_name, salary * 1.1
from instructor;
```

The where Clause

- The **where** clause specifies conditions that the result must satisfy
  - Corresponds to the selection predicate of the relational algebra.
- To find all instructors in Comp. Sci. dept

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- SQL allows the use of the logical connectives **and, or,** and **not**
- The operands of the logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>.
- Comparisons can be applied to results of arithmetic expressions
- To find all instructors in Comp. Sci. dept with salary > 80000

```
select name
from instructor
where dept_name = 'Comp. Sci.' and salary > 80000
```

Find the names of all instructors in the Computer Science department who have salary greater than 80,000.

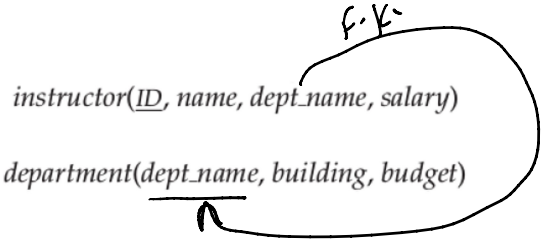
The where clause allows us to select only those rows in the result relation of the from clause that satisfy a specified predicate.

The from Clause

- The **from** clause lists the relations involved in the query
  - Corresponds to the Cartesian product operation of the relational algebra.
- Find the Cartesian product *instructor X teaches*

```
select *
from instructor, teaches
```

  - generates every possible instructor – teaches pair, with all attributes from both relations.
  - For common attributes (e.g., *ID*), the attributes in the resulting table are renamed using the relation name (e.g., *instructor.ID*)
- Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).



department(*dept_name*, *building*, *budget*)

- Retrieve the names of all instructors, along with their department names and department building name.

```
select name, instructor.dept_name, building
from instructor, department
```

```
select name, instructor.dept_name, building
from instructor, department
where instructor.dept_name= department.dept_name;
```

Retrieve the names of all instructors, along with their department names and department building name

```
select name, instructor.dept_name, building
from instructor, department
where instructor.dept_name= department.dept_name;
```

Note that the attribute *dept_name* occurs in both the relations *instructor* and *department*, and the relation name is used as a prefix (in *instructor.dept_name*, and *department.dept_name*) to make clear to which attribute we are referring. In contrast, the attributes *name* and *building* appear in only one of the relations and therefore do not need to be prefixed by the relation name.

The General Form

```
select A1, A2, ..., An
from r1, r2, ..., rm
where P;
```

```
for each tuple t1 in relation r1
        for each tuple t2 in relation r2
                ...
                for each tuple tm in relation rm
                        Concatenate t1, t2, ..., tm into a single tuple t
                        Add t into the result relation
```

Example

instructor(<u>*ID*</u>, *name*, *dept_name*, *salary*)

teaches(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>)

- For all instructors in the Computer Science department who have taught some course, find their names and the course ID of all courses they taught.

3 **select** *name*, *course_id*
1 **from** *instructor*, *teaches*
2 **where** *instructor.ID*= *teaches.ID* **and** *instructor.dept_name* = 'Comp. Sci.';

Steps of Execution

1. Generate a Cartesian product of the relations listed in the **from** clause.
2. Apply the predicates specified in the **where** clause on the result of Step 1.
3. For each tuple in the result of Step 2, output the attributes (or results of expressions) specified in the **select** clause.